

SAKI: Shared Adaptive KV-Instance Control under a Fixed Compaction Budget

Qichu Tian

Xi'an Jiaotong University
mizukiqwq@stu.xjtu.edu.cn

Shipeng Zhao

Xi'an Jiaotong University
zhaoshipeng@stu.xjtu.edu.cn

Heng Chen

Xi'an Jiaotong University
hengchen@xjtu.edu.cn

Bo Wang

Xi'an Jiaotong University
Aroom@stu.xjtu.edu.cn

ABSTRACT

Multi-tenant deployments often co-locate many RocksDB instances on one host. Each tenant has an independent LSM pipeline, but flushes and compactions draw from a shared background I/O and compaction budget. When write pressure drifts, equal per-instance allocation can leave service stranded at quiet tenants while tenants creating compaction debt suffer high write-tail latency. We frame this as fixed-budget placement: moving the existing background budget toward tenants where compaction pressure is forming.

SAKI is an external controller for this placement problem. In its conserved rank-and-assign framework, the controller observes per-tenant demand and pressure symptoms, ranks tenants without oracle labels or workload forecasts, and reallocates per-tenant background budgets while preserving the aggregate budget. A demand-observability model explains when demand-aware score instances track bounded drift within one window and why pressure-only ranking can suffer stale-debt lag. SAKI requires no changes to RocksDB's compaction algorithm or tenant application code. Across five 16-tenant epoch-level drift trials, SAKI reduces high-pressure write P99 by 11.7%, raises high-pressure throughput by 38.1%, and lowers compaction write bytes by 21.0% relative to equal static allocation, while keeping total throughput roughly neutral. In a continuous embedded RocksDB harness with long-running tenants and runtime actuation through `RateLimiter::SetBytesPerSecond()`, SAKI reduces high-pressure write P99 by 27.0%, reduces P999 by 47.1%, raises high-pressure throughput by 26.1%, and lowers normalized bytes/write by 11.9%. Stronger baselines, ranking replay, score-family perturbations, and mechanism checks attribute these gains to online placement rather than extra capacity, static skew, score fragility, or unrelated throttling. Public-trace audits identify what future production traces must preserve for controller validation. The results show that shared compaction budgeting is a practical host-level control point for multi-tenant RocksDB deployments under fixed background budgets.

PVLDB Reference Format:

Qichu Tian, Heng Chen, Shipeng Zhao, and Bo Wang. SAKI: Shared Adaptive KV-Instance Control under a Fixed Compaction Budget. PVLDB, 20(1): XXX–XXX, 2027.
doi:XX.XX/XXX.XX

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at <https://github.com/M1zukiqwq/PVLDB2027-SAKI-OpenSource>.

1 INTRODUCTION

Modern data-serving systems often place many storage-engine tenants on one host. Each tenant may own a separate RocksDB instance, data directory, configuration, and foreground workload, yet their flushes and compactions draw from the same finite background CPU and storage bandwidth. Per-instance controls are useful, but they do not decide how a fixed host budget should be divided across competing instances.

This placement decision becomes a performance problem when demand drifts. Equal static allocation leaves capacity assigned to quiet tenants; frozen skew becomes stale when the high-pressure set moves; and RocksDB's per-instance auto-tuned rate limiter adapts within a configured ceiling but cannot move budget across instances. The resulting compaction-debt mismatch appears as high write-tail latency and lower completion rates for tenants currently creating debt.

We study the host-level placement decision directly. Under a fixed aggregate background compaction/I/O budget, can an online controller use runtime observations to reallocate per-tenant budgets across co-located RocksDB tenants, improving the tenants currently under write pressure without oracle labels, workload forecasts, or extra host capacity? The question is not whether more compaction capacity helps, but whether the same capacity can be placed better than fixed fair sharing.

SAKI is an external controller for this placement problem. It observes each tenant over a control window, ranks tenants using offered write demand and compaction-pressure symptoms, and applies the next-window budget assignment while conserving the aggregate budget. This conserved rank-and-assign framework separates the placement invariant from the score instance. Its key condition is demand observability: when offered demand separates currently high-pressure tenants from the rest, a demand-aware score family tracks bounded drift within one control window, while pressure-only ranking can chase stale debt. SAKI sits outside RocksDB; in the continuous harness, it changes runtime rate-limiter budgets through `RateLimiter::SetBytesPerSecond()`.

licensed to the VLDB Endowment.

Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

The evaluation follows an attribution chain. It first asks whether SAKI improves the target tenants under the same aggregate budget, using both epoch-level and continuous RocksDB experiments. It then tests alternative explanations, including static skew, tenant-local autotuning, stale-debt scoring, lower-layer I/O weights, and foreground throttling. Finally, ranking-margin replay, score-family perturbations, workload-matrix experiments, collateral measurements, liveness checks, and public-trace audits define where the evidence applies.

In a 16-tenant epoch-level workload, five trials show that SAKI reduces high-pressure write P99 by 11.7%, raises high-pressure throughput by 38.1%, and lowers compaction write bytes by 21.0% relative to equal static allocation, with roughly neutral total throughput. In a continuous embedded RocksDB harness with run-time actuation, five trials show that SAKI reduces high-pressure write P99 by 27.0%, reduces P999 by 47.1%, raises high-pressure throughput by 26.1%, and lowers normalized bytes/write by 11.9%. The tradeoff is visible: lower-pressure tenants may absorb part of the cost, so SAKI is a targeted reallocation policy.

The paper makes five contributions:

- (1) It frames multi-tenant RocksDB compaction as a fixed-budget host-level reallocation problem, distinct from single-instance compaction-policy design.
- (2) It presents SAKI, a conserved rank-and-assign controller that sets per-tenant runtime budgets from previous-window observations without oracle labels, workload forecasts, or tenant application changes.
- (3) It identifies the demand-observability condition that lets a demand-aware score family track bounded drift, and contrasts it with the stale-debt lag of pressure-only ranking.
- (4) It provides epoch-level and continuous evidence that online reallocation improves high-pressure write latency and throughput under bounded drift while keeping total throughput close to static allocation.
- (5) It separates SAKI’s effect from fixed skew, per-instance local autotuning, score-instance choice, workload variation, and lower-layer I/O control, while making collateral costs and validation boundaries explicit.

Together, these results show that shared host-level compaction budgeting is a practical control point for multi-tenant RocksDB write performance under bounded drift.

2 BACKGROUND AND MOTIVATION

SAKI targets co-located RocksDB instances whose background work shares finite host resources. RocksDB exposes useful local controls, but not the host-wide placement decision: how that fixed service budget should be placed across instances as tenants’ write pressure changes at different times.

2.1 LSM Background Work

The log-structured merge-tree makes foreground writes inexpensive by buffering updates and later merging them into on-disk components [22]. Compaction rewrites sorted files, removes obsolete entries, controls level sizes, and bounds read cost. This design gives high write throughput but turns recent foreground writes into future CPU and I/O demand. That deferred demand becomes

a host-level problem when several engines share one finite background budget; surveys and recent microbenchmarks emphasize compaction, amplification, and buffering as central LSM trade-offs [12, 16].

RocksDB exposes per-instance background jobs, rate limiters, statistics, properties, and event listeners [18]. SAKI uses them as the measurement and actuation surface for the host-level placement problem.

2.2 From Tenant-Local State to Host-Level Debt

We use *compaction debt* as an operational term for deferred background work and its foreground consequences. Pending compaction bytes, L0 files, flush/compaction events, write-tail latency, and completion gaps are observable but incomplete symptoms that help identify tenants currently creating deferred maintenance.

The cross-tenant aspect is the key. Under a fixed aggregate budget, equal allocation leaves capacity assigned to quiet tenants whenever only a subset is write-heavy. Frozen skew helps only while the workload resembles the phase it was tuned for. Section 7 tests this mismatch between where the budget sits and where debt is being created.

Compaction service has delayed effects: too little service may surface later as L0 pressure, flush backlog, write slowdown, or tail latency; too much service for a quiet tenant consumes capacity that could reduce risk elsewhere. The evaluation therefore reports high-pressure latency and throughput together with total throughput and collateral impact on lower-pressure tenants.

2.3 Why Existing Directions Do Not Settle the Question

Prior LSM work changes single-engine policies or storage layout: Monkey, Dostoevsky, WiscKey, PebblesDB, and SILK improve what an engine does given an assigned service budget [2, 4, 5, 15, 23]. SAKI asks where a fixed host budget should be placed across independent engines.

CaaS-LSM treats compaction as a service and DFlush offloads flush work to DPUs [6, 35]. SAKI shares the resource-management view, but targets co-located RocksDB instances on one host, standard runtime controls, and a conserved aggregate budget. The next section formalizes that host-level allocation question.

3 PROBLEM DEFINITION

We study a host with tenants $T = \{1, \dots, N\}$. Each tenant is an independently configured RocksDB instance with its own memtables, WAL, SST files, flush path, compaction path, and foreground workload. Tenants do not share logical key spaces, but their background work draws from shared host resources. Time is divided into control windows $w = 1, 2, \dots$. In each window, the controller assigns tenant i a background budget $b_i(w)$. The unit of this budget is harness-dependent: epoch runs change rate limits and background-job settings between epochs, while continuous runs keep tenants alive and change the rate-limiter budget through `RateLimiter::SetBytesPerSecond()`.

The aggregate budget is fixed. If B is the host budget, a valid assignment must satisfy $\sum_i b_i(w) = B$, subject to the discrete budget tiers exposed by the implementation. Without this conservation

rule, a controller could improve high-pressure tenants simply by spending more host capacity.

The controller is online. At the start of window w , it may use only observations from windows up to $w - 1$. The observation vector $x_i(w - 1)$ can include demand signals such as offered write demand and completed write throughput; pressure symptoms such as completion gap, write-tail latency, pending-compaction proxies, and L0 pressure; background-work signals such as compaction bytes and flush/compaction events; read-side measurements; and the prior assignment. It may not include the next high-pressure set, future phase id, or evaluator-only tier labels. SAKI also does not observe a single ground-truth scalar called compaction debt; it observes symptoms of deferred work and foreground pressure.

The objective is targeted relief for tenants currently under high write pressure, measured by write-tail latency and write throughput, while preserving attribution to online reallocation under a fixed budget. Because reallocation is a tradeoff, collateral effects must also be visible: the evaluation reports mid- and low-pressure effects, total throughput, and liveness failures. Each decision therefore chooses $b_i(w)$ from implementable budget levels using $x_i(w - 1)$, conserving B under delayed feedback.

Several constraints sharpen the scope. Policies cannot read workload labels or future drift schedules. Continuous runs actuate only the runtime rate limiter, rather than every RocksDB option. Assignments occur at window boundaries, so very fast drift may outrun a previous-window controller. SAKI is therefore not a new compaction algorithm, an in-engine global scheduler, or an oracle classifier. The definition leaves the ranking rule open; the next section instantiates this online fixed-budget problem with a conserved rank-and-assign framework and a transparent demand-aware score family.

4 SAKI DESIGN

SAKI is a host-level closed-loop controller that implements a conserved rank-and-assign framework, sketched in Figure 1. The framework is the method contribution: it preserves a fixed-budget invariant and assignment template while allowing different observable score instances to rank tenants. SAKI sits outside RocksDB, observes each tenant over window w , assigns budget tiers for window $w + 1$, and validates that the aggregate budget is unchanged. The one-window delay is intentional: signals from window w must remain informative for window $w + 1$.

4.1 Conserved Rank-and-Assign Framework

The framework decouples a budget-conserving assignment template from the score instance used to rank tenants. The control loop, illustrated by the four pipeline steps in Figure 1, is intentionally compact:

Input: $\{x_i(w)\}_{i=1}^N, B, b_H, b_M, b_L, H, L$
Score: $s_i(w) \leftarrow \text{score}(x_i(w))$
Rank: sort tenants by descending $s_i(w)$
Assign: top $H \rightarrow b_H$, bottom $L \rightarrow b_L$, others $\rightarrow b_M$
Check: $\sum_i b_i(w + 1) = B$
Actuate: apply $b_i(w + 1)$ for the next window.

The tiered assignment makes the policy auditable. We distinguish *high-pressure* tenants, which the workload currently stresses, from

high-budget tenants, which SAKI assigns to b_H . A tier configuration is budget-conserving when $H(b_H - b_M) = L(b_M - b_L)$; the continuous main setting uses $4(11 - 6) = 4(6 - 1)$, yielding the same 96 MB/s aggregate budget as sixteen all-mid tenants. Other tier counts or continuous allocations can use the same rule; we fix auditable tierings to isolate the placement mechanism within each experiment family.

The score instance instantiates the framework; it is not the framework itself. This separation lets the paper make a framework claim and then test a small family of auditable instances under the same conserved assignment rule. The reference instance below is intentionally simple so the evaluation attributes gains to online placement under conservation, not to an opaque learner or extra capacity.

4.2 Observations and Labels

The observation vector is tenant-local, but the decision is host-wide. In the continuous harness, $x_i(w)$ includes offered write/read rates, completed operations, write latency percentiles, compaction and flush bytes from an event listener, estimated pending compaction bytes, L0 files, memtable bytes, delayed-write rate, write-stop status, and the current assignment. These are runtime measurements rather than workload labels.

The experiment generator retains each tenant’s synthetic workload tier so the evaluation can compute high-pressure benefit and high-set overlap. The controller does not read that tier, a future phase identifier, or the next high-pressure set. Offered write demand is an observed admission-side signal from the previous window, rather than a prediction.

4.3 Reference Score Instance and Family

SAKI supports several score instances for ablation studies. The main continuous policy uses a demand-aware reference instance. Let D_i be offered write QPS, C_i completed write QPS, $G_i = \max(0, D_i - C_i)$ the completion gap, B_i the compaction output in MB, P_i pending compaction bytes in MB, and L_i the number of L0 files capped at 6. The bounded tail term is

$$T_i = \min(P99_i/1000, 80) + \min(P999_i/5000, 40),$$

where latency is measured in microseconds. In demand mode, SAKI scores tenant i as

$$s_i = 6.00D_i/1000 + 0.60G_i/1000 + 0.05T_i + 0.02B_i + 0.02P_i + 0.30L_i.$$

The coefficients reflect a design choice, not a universal optimum. Offered write demand anchors active high-pressure tenants, while bounded residual terms—completion gap, tail latency, compaction output, pending bytes, and L0 pressure—refine near-boundary tenants. This division targets two failures: pressure-only scores can chase stale debt after demand moves, while demand-only scores lack a maintenance signal for tenants that have just left a high-demand phase. Residuals should not overturn a clearly separated demand signal, but can stabilize rankings when margins are small. The evaluation tests this with offline decision replay and online co-efficient perturbations; in the main continuous workload, replaying all 35 post-warmup decisions shows that the demand term alone reproduces the full reference score’s top-four assignment.

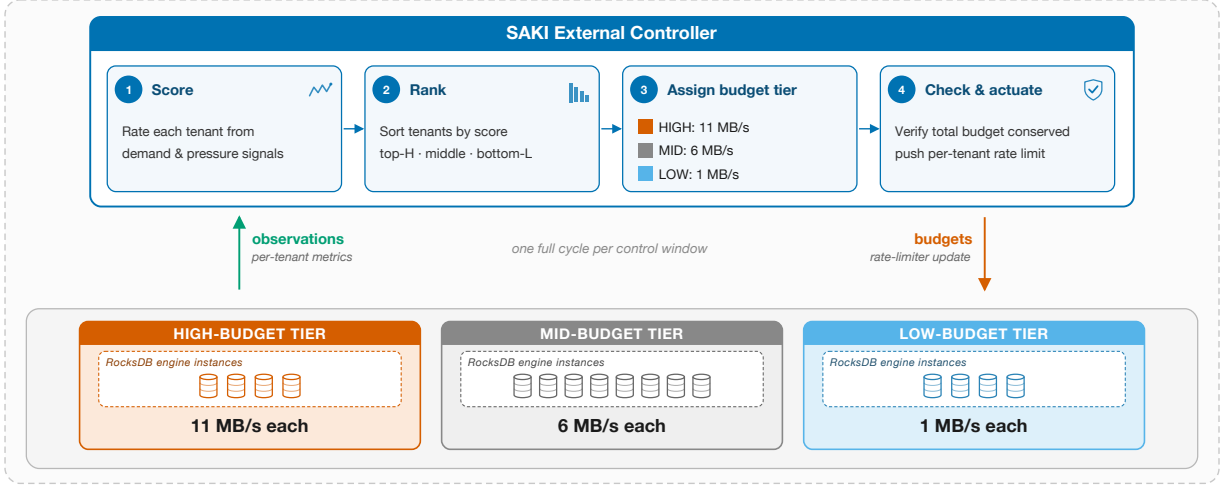


Figure 1: SAKI conserved rank-and-assign loop. Each window, the external controller scores tenants from previous-window demand and pressure signals, ranks them, assigns high/mid/low budget tiers, checks that the aggregate host budget is preserved, and pushes runtime rate-limit updates. The shown 4:8:4 split at 11:6:1 MB/s is the main continuous setting.

The score is fixed and auditable, but SAKI remains online: observations, ranking, and assignment are recomputed every window. Keeping the rule fixed separates the placement question from learning coefficients under delayed, partially counterfactual feedback. The relevant design space is a family of demand-aware instances whose primary term is offered write demand and whose secondary pressure terms are bounded refinements. Section 7 therefore tests demand-only replay, online anchor-scaled, residual-scaled, and anchor-only instances, along with pressure-only negative controls, under the same rank-and-assign template. This design yields two falsifiable predictions: demand-aware instances should track moving pressure with high overlap and low lag, while pressure-only instances should expose stale-debt lag. The coefficients are not portable constants; deployments could add normalization or policy layers while preserving budget conservation and demand observability.

The epoch harness uses a coarser, pressure-oriented score because each epoch exposes clearer symptoms. The continuous harness is stricter: decisions are made every short window while tenants stay active. Demand mode is the main continuous policy; pressure and hybrid modes serve as ablations.

4.4 Actuation and Stability

The actuation path matches the harness. Epoch-level runs can change background-job and rate-limit settings between epochs. Continuous runs keep each tenant active and adjust only the runtime rate-limiter budget; `max_background_jobs` remains fixed at process launch. Section 5 gives the specific runtime API.

SAKI’s framework-level stability rests on budget conservation, bounded score terms, tiered assignment, a low-tier floor, and one-window updates. We evaluate this envelope through high-set overlap, adaptation lag, total throughput, bytes/write, and tier-level collateral effects. Production deployments could add hysteresis,

cooldowns, minimum-dwell rules, or SLO-weighted floors. Per window, the controller aggregates N tenant observations, sorts tenants by score, validates the conserved budget, and issues N budget updates. The control-plane work is $O(N \log N)$ and stays outside the foreground write path. The paper does not include an overhead-scaling sweep at larger tenant counts.

4.5 Analytical Motivation

The design choices in the preceding subsections are motivated by a simple analytical model. The model is not an optimizer for the coefficients; instead, it isolates the tracking condition that the score family must satisfy and explains the experimental divergence between demand-mode and pressure-mode controllers reported in Section 7.

Model. Let tenant i have offered write demand $d_i(w)$, completed write rate $c_i(b)$ under budget b , and normalized compaction debt $q_i(w)$ in equivalent writes. Budget tiers satisfy $b_H > b_M > b_L$ with conservation $Hb_H + (N - H - L)b_M + Lb_L = B$. Backlog evolves as

$$q_i(w + 1) = \max(0, q_i(w) + T(d_i(w) - c_i(b_i(w))))$$

where T is the window length. We call tenant i *high-pressure* when $d_i(w)$ exceeds the service rate achievable under b_M ; the deficit under b_L is larger still.

Why static allocation can fail. Under equal allocation $b_i = B/N$, if a high-pressure tenant satisfies $d_i > c_i(B/N)$, then q_i grows without bound until a write stall or SLO violation occurs. This is the baseline problem: equal sharing allocates the fixed budget without regard to which tenants are currently creating debt.

Proposition 1 (demand-aware one-window tracking). Consider a score instance in the rank-and-assign framework whose ordering can be written as $s_i = \alpha D_i(w) + \rho_i(w)$, where $D_i(w)$ is the observed offered demand and the residual term $\rho_i(w)$ is computed from bounded pressure and maintenance signals. Suppose the high-pressure set to be served has at most H tenants and remains active

into the next control window. If, in window w , every such tenant has offered demand at least Δ above every non-high tenant and

$$\alpha\Delta > \max_{i,j} |\rho_i(w) - \rho_j(w)|,$$

then the framework places those tenants in the high-budget tier in window $w + 1$.

The proof is immediate from the ordering condition: for any high-pressure tenant i and non-high tenant j , $s_i - s_j > 0$, so the top- H rank contains the high-pressure set. The one-window delay is unavoidable because the controller only observes $D_i(w)$ after window w completes. The condition is sufficient, not universal: in the continuous main trials, offline replay shows that the demand term alone produces the same top-four assignment as the reference score in all 35 post-warmup decisions.

Proposition 2 (stale-debt lag for pressure-only ranking). Consider a monotone pressure-only instance that ranks tenants by backlog or maintenance signals but does not observe offered demand. Immediately after a workload drift, the instance cannot distinguish an exiting tenant with historical debt from an entering tenant whose current demand is forming fresh debt. If the exiting tenant starts the post-drift period with residual pressure equivalent to q writes above the entrant, and the extra drain rate gained by removing its high-budget slot is at most $\Delta c = c(b_M) - c(b_L)$, then the stale high-budget assignment can persist for $\Omega(q/(T\Delta c))$ windows under a monotone backlog-priority rule.

This is not a convergence theorem for all controllers; it is a structural ambiguity in the pressure-only family. It predicts the measured divergence between score instances: demand mode tracks with overlap 3.00/4 and zero max lag, while pressure-only mode has overlap 1.86/4 and max lag 5 windows. The coefficients remain a deployment choice, but the demand-observability condition is the mechanism that separates tracking from stale-debt chasing.

5 IMPLEMENTATION

SAKI is implemented as an external Python controller plus two experiment harnesses around unmodified RocksDB instances. The implementation separates the paper’s two control questions and realizes the conserved rank-and-assign framework from Section 4. The epoch harness asks whether a fixed budget can be placed better between workload phases; the continuous harness asks whether the same placement loop can actuate a live RocksDB rate limiter while tenants keep running. In both paths, the controller chooses high, mid, and low budget assignments, the harness applies them through ordinary RocksDB-facing APIs, and the measurement path records per-tenant behavior at the same granularity used by the controller.

5.1 Epoch Harness

The epoch-level path uses `db_bench` as the tenant process. A trial creates one database directory per tenant, runs a prefill phase, and then executes a sequence of workload epochs. Each epoch assigns tenants to high, mid, and low demand tiers, launches one `db_bench` process per tenant, parses the logs, and uses the resulting observations for the next epoch’s allocation decision. Because every epoch is a fresh invocation, this harness can vary both `max_background_jobs` and `rate_limiter_bytes_per_sec`

at epoch boundaries. The driver checks that the resulting job and rate-limit settings match the static fair-share budget before launch.

The parser extracts per-tenant runtime, exit status, operation rate, latency histograms where available, compaction read/write bytes, flush bytes, keys and bytes written, and compaction CPU time. It stores these rows with the epoch, workload tier, assigned budget tier, background-job count, rate limit, and workload parameters. This path gives clean mechanism evidence for budget placement under drift because all policies run comparable foreground work under the same aggregate budget. It does not by itself prove in-process runtime control; that is the role of the continuous harness.

5.2 Continuous Harness

The continuous path is a custom C++ embedded RocksDB harness. Each tenant is a long-running process linked against RocksDB. The Python driver writes one control file per tenant, advances through fixed-length windows, computes the current workload tier and policy assignment, validates the aggregate byte budget, and atomically updates the control file. Tenant processes keep running while their offered write/read rates, hot-key range, reported workload tier, assigned budget tier, and byte budget are refreshed. Before each refresh the driver checks that the next assignment preserves the aggregate byte budget. Control files are written atomically, and a tenant keeps using its last installed budget until it observes a newer value. Thus a missed or delayed update does not create extra host budget in the harness; a production controller would additionally fall back to all-mid allocation when tenant inventory or metrics are incomplete.

Runtime actuation uses RocksDB’s rate limiter. At startup, each tenant creates a `NewGenericRateLimiter` with the initial budget and installs it before opening the database. During the run, a control thread rereads the budget and calls `RateLimiter::SetBytesPerSecond()` on the same rate-limiter object. This is the main continuous actuation path. Background-job count is a launch-time option and remains fixed for the tenant process. Diagnostic userspace throttling exists in the harness but is not the mechanism used for the reported continuous result.

The rate limiter is a RocksDB-facing background-I/O control rather than an application-level admission throttle. In the reported setup it is installed as the database rate limiter and therefore shapes the I/O issued by flush and compaction work that uses RocksDB’s rate-limited paths. SAKI does not directly sleep foreground write calls or read calls. Foreground write latency changes indirectly because background service changes the accumulation and draining of LSM maintenance work. This distinction makes the continuous result a maintenance-placement result rather than a foreground-throttling result: SAKI changes the maintenance budget available to each tenant, not the offered foreground workload. In the five main continuous trials, rate-limiter, compaction, and flush byte deltas are larger in high-budget windows than in low-budget windows. As an internal-validity check on this actuation path, we also ran a homogeneous four-tenant static microstudy that sweeps the per-tenant limiter budget $B \in \{1, 3, 6, 11\}$ MB/s with three seeds per point. Dropping the first window, measured rate-limiter traffic is monotone in B : 1.00, 2.96, 5.70, and 7.28 MB/s. Compact-plus-flush output moves with the limiter (coverage ratio 0.99–1.02); the

11 MB/s point saturates at about 7.3 MB/s because this workload cannot generate more maintenance I/O. A two-seed no-rate-limiter sanity run reports zero limiter bytes while background maintenance continues near 8 MB/s. The microstudy shows that the continuous knob controls background-maintenance I/O rather than foreground throttling.

The embedded harness records per-window write and read operation counts, throughput, and latency percentiles. A custom `EventListener` accumulates compaction input/output bytes, compaction CPU time, compaction count, flush bytes, and flush count. At window boundaries the harness emits deltas, together with RocksDB properties such as pending compaction bytes, L0 files, memtable bytes, delayed-write rate, and write-stop status. These measurements feed both the online score and the post-run analysis.

5.3 Analysis

The analysis scripts consume JSON payloads produced by the drivers. For continuous runs, the analyzer merges tenant metrics, allocation history, process exit codes, commands, run parameters, and summary counters. It computes write P99/P999 and throughput for high-pressure tenants, total throughput, compaction output bytes, bytes/write, high-set overlap, adaptation lag, and failed-tenant counts. The same pipeline emits the paper-table summaries and the time-series data used for the evaluation figures. This keeps the manuscript numbers tied to generated artifacts rather than manual transcription.

6 EXPERIMENTAL METHODOLOGY

The experiments are organized around the fixed-budget placement claim. The first layer asks whether moving a host-level background budget toward tenants currently carrying write pressure improves their write path without extra capacity. The second layer narrows the mechanism: runtime actuation tests deployable control, stronger baselines test fixed skew and local autotuning, score instances test the adaptation signal, ranking replay and score-family perturbations test robustness, cgroups test a lower-layer control point, and the rate-limiter coverage microstudy checks the maintenance-I/O knob. The third layer reports workload coverage, tier-wise collateral cost, liveness, and trace qualification boundaries. Budget conservation is enforced within each experiment family. The epoch-level and public-trace qualification runs use an 11/7/3 MB/s, 112 MB/s contract; the continuous runtime family uses an 11/6/1 MB/s, 96 MB/s contract. These numbers differ because the harnesses expose different actuation surfaces, but every SAKI comparison holds the relevant family budget fixed against its baselines.

6.1 Workloads

The epoch-level anchor is EPOCH-DRIFT16. It uses 16 co-located tenants, three demand tiers, eight epochs, and gradual movement of a four-tenant high-pressure set. Tenants are prefilled before the measured epochs.

The continuous anchor is RUNTIME-DRIFT16. It also uses 16 tenants and three tiers, but runs tenants as long-lived embedded RocksDB processes for eight 20-second windows. The high-pressure set moves by one tenant per window, so the experiment tests on-line adaptation at a rate the controller can plausibly follow. The

continuous anchor uses 11/6/1 MB/s high/mid/low budgets with a 4/8/4 split, preserving a 96 MB/s aggregate against an all-mid static baseline. A window-length sensitivity bracket repeats the same anchor at 10, 20, and 40 seconds for three static/online seed pairs per point, holding the 160-second duration and all per-tenant budgets fixed. A score-family robustness ablation perturbs the demand-mode instance along anchor weight and a uniform residual scale ($\times 0.5$, $\times 2$, and $\times 0$ meaning anchor-only), with three seeds per row paired with paper-static a/b/c; results appear in Section 7. We also replay all five continuous main online traces offline to measure the ranking margin and the top- H invariance of nearby score instances without launching additional RocksDB runs. A separate 360-second longer-run is a sanity check for obvious instability.

We also run a local sensitivity sweep around EPOCH-DRIFT16, changing drift speed (`epochs_per_stage`=3 and 1, versus 2) and high write pressure (70k and 130k writes/s, versus 100k). Each new point starts with two static/SAKI screening trials; stress-side variants are promoted to five trials if a pre-committed rule holds (high P99 improves by at least 2%, high throughput by at least 5%, overlap $\geq 2.5/4$, static does not hit capacity-boundary).

Around RUNTIME-DRIFT16 we run a controlled workload matrix that perturbs one axis at a time: VALUE-4K uses 4 KB values, READ-3x triples per-tier point-read QPS, FAST-DRIFT moves the high-pressure set at 1.5 tenants per window, and TENANT-8 uses a smaller 8-tenant cluster with two high-pressure tenants. Per-tenant budgets are preserved so the same-budget contract holds; for TENANT-8 the aggregate scales linearly. The matrix uses the paper SAKI policy without changing the score instance. Every variant is promoted from a two-trial screen to five trials unless either policy has a failed tenant, and the overlap threshold is rescaled to each variant’s high-pressure tenant count. The 20-second window, tier budgets, and 4/8/4 split are mechanism-isolation settings, not portable constants. The 4/8/4 split is chosen because the high-budget tier’s surplus exactly matches the low-budget tier’s savings, preserving the same aggregate budget within each experiment family; Section 4 states the general conservation rule. FAST-DRIFT and TENANT-8 are local sensitivity, not a parameter search.

6.2 Policies

static is the main fair baseline: every tenant receives the same share of the aggregate background budget for that experiment family. SAKI is the online policy: it assigns high, mid, and low tiers from previous-interval observations while preserving that aggregate budget. The main continuous SAKI policy uses the demand-aware reference score instance.

`static_biased` is a frozen-skew baseline for the epoch workload. It uses the same high/mid/low budget tiers, but freezes the initial-stage assignment for all later epochs. It models an operator who pre-tunes a reasonable non-equal allocation and then does not adapt as the high-pressure set drifts.

`static_autotuned` is the strict same-budget built-in RocksDB baseline. It uses exactly the same per-tenant ceilings as `static`. In the epoch configuration this is two background jobs and 7 MB/s per tenant, for a 32-job and 112 MB/s aggregate budget across 16 tenants. Each tenant enables `-rate_limiter_auto_tuned=1`, so adaptation

stays local to that RocksDB instance and remains within the configured per-instance range. This baseline asks whether per-instance local rate adaptation can replace the outer host-level placement decision; we keep the ceilings identical to static. For the sensitivity sweep, `static_autotuned` is added only to the promoted higher-pressure point.

`oracle_tiered` is a diagnostic reference using true tier information. `unlimited` is a capacity reference, not a same-budget competitor.

6.3 Platform

All experiments run on a single Linux machine with 80 logical cores (Ubuntu, kernel 6.17). Storage is a local SSD. Each RocksDB tenant writes to its own data directory on the same device. The epoch-level harness uses `db_bench 8.9.1` from unmodified Ubuntu .deb archives; the continuous harness links against the same RocksDB version. No other major I/O-intensive processes run during measured trials.

6.4 Metrics and Aggregation

Primary metrics are write P99/P999 latency and write throughput for high-pressure tenants, the foreground symptoms SAKI is meant to improve by moving background service. Total throughput shows whether the host collapses aggregate work under the fixed budget. Tier-wise latency and throughput expose the redistribution cost that a targeted policy can impose on lower-pressure tenants. The continuous rate limiter is background-I/O control: it shapes RocksDB flush/compaction I/O, so foreground changes reflect maintenance placement rather than direct sleeps on application reads or writes.

Epoch-level efficiency reports compaction write bytes. In continuous runs SAKI changes foreground completion, so the main compaction efficiency metric is normalized bytes/write; absolute compaction bytes are reported but not the main efficiency claim. Epoch runs execute comparable foreground work per trial, so absolute background bytes are directly interpretable; continuous runs may complete more foreground writes under SAKI and need per-write normalization.

Adaptation is measured by high-set overlap (high-pressure tenants assigned the high-budget tier after warmup) and adaptation lag in control windows. Ranking robustness is measured by replaying the previous-window observations that fed each online decision and comparing the resulting top- H set across score instances using exact-match count and Jaccard similarity. Liveness is measured by failed tenant processes. The main epoch and continuous results aggregate five independent trials. Percent changes are computed relative to static within each trial before aggregation. The paper reports mean, range, standard deviation, and confidence intervals. Sensitivity rows with two trials report a mean and range; five-trial rows report 95% CI half-widths. Raw and summarized artifacts are recorded under `remote-results/`, and figures use real result files from the figure data manifest.

Table 1: Epoch-level main result, EPOCH-DRIFT16 (online vs. static).

Metric	Mean	Range	95% CI
High write P99	-11.7%	[-17.2,-6.2]	[-18.0,-5.5]
High throughput	+38.1%	[+34.2,+42.1]	[+34.4,+41.7]
Total throughput	-1.7%	[-4.7,+2.1]	[-4.9,+1.4]
Compact bytes	-21.0%	[-24.3,-18.4]	[-23.6,-18.4]
Mean high overlap	3.14/4	[3.14,3.14]	–

6.5 Scope

The methodology supports a constrained claim: fixed-budget online reallocation in two controlled RocksDB drift regimes. The built-in local rate-limiter baseline is covered under the same aggregate budget because it tests whether tenant-local adaptation can replace the host-level rank-and-assign loop. OS/cgroup I/O controls are included for a different reason: they can enforce weights or throttles below RocksDB, but they do not observe LSM debt signals or decide which engine should receive compaction service. We include a 5-trial aligned cgroup-v2 baseline at the same scale as the epoch main result (16 tenants, 8 epochs, 240k keys/tenant, 100k writes/high tier) as supplemental mechanism evidence, not as a repeated main baseline. The controller inside the cgroup harness uses a simplified pressure score for cross-mechanism comparison; it is not the main SAKI controller used in the primary RocksDB experiments.

7 EVALUATION

The evaluation follows the paper’s fixed-budget claim through three questions. First, within each harness’s conserved budget contract, can RocksDB background service do more useful work when placed on tenants currently creating compaction debt? Second, is the effect online host-level placement rather than extra capacity, static skew, local autotuning, a stale backlog signal, or a lower-layer throttle? Third, what workload coverage, collateral cost, and trace-qualification boundary remain after the mechanism is established? Unless stated otherwise, percentages are online SAKI versus equal static allocation; negative latency or bytes/write values are improvements, and positive throughput values are improvements. In metric labels, “High” refers to the workload’s high-pressure tenants; high-set overlap measures how often those tenants receive SAKI’s high-budget assignment. All reported RocksDB and cgroup runs completed with zero failed tenants.

7.1 Fixed-Budget Placement Improves High-Pressure Tenants

The epoch-level harness isolates placement by changing RocksDB settings only between epochs. Across five repeated 16-tenant drift trials, SAKI reduces high-pressure write P99 by 11.7%, raises high-pressure throughput by 38.1%, and reduces compaction write bytes by 21.0% (Table 1). Total throughput remains roughly neutral: the mean is -1.7%, and its CI crosses zero. All five trials improve high-pressure write P99, high-pressure throughput, and compaction bytes; mean high-set overlap is 3.14/4, so the policy places high budget near the moving high-pressure set without using oracle labels.

Table 2: Continuous main result, RUNTIME-DRIFT16 (five trials, online vs. static). CI half-widths in pp.

Metric	Mean	Min	Max	95% CI
High write P99	-27.0%	-39.2%	-12.0%	$\pm 12.5\text{pp}$
High write P999	-47.1%	-54.0%	-39.3%	$\pm 8.8\text{pp}$
High write throughput	+26.1%	+21.5%	+28.2%	$\pm 3.5\text{pp}$
Total throughput	+4.3%	+1.3%	+6.8%	$\pm 3.2\text{pp}$
Compact bytes	-1.9%	-5.8%	-0.1%	$\pm 2.9\text{pp}$
Bytes/write	-11.9%	-13.5%	-9.1%	$\pm 2.3\text{pp}$
Mean high overlap	3.00/4	3.00	3.00	–

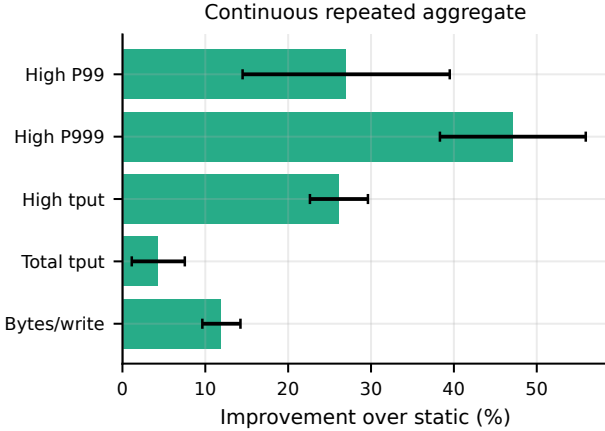


Figure 2: Continuous repeated aggregate. Each bar is normalized so positive means improvement over equal static; error bars show 95% CI half-widths.

The continuous harness tests the stricter deployment boundary: tenants remain alive, workload phases drift every 20 seconds, and SAKI changes only the runtime rate-limiter budget through `RateLimiter::SetBytesPerSecond()`. Table 2 and Figure 2 summarize five RUNTIME-DRIFT16 trials. SAKI reduces high-pressure write P99 by 27.0%, P999 by 47.1%, and raises high-pressure write throughput by 26.1%. Because SAKI completes more foreground writes, normalized bytes/write is the main continuous efficiency metric; it falls by 11.9%, while absolute compact bytes change only -1.9%. The continuous result shows the placement effect is not an epoch artifact: the same conserved-budget loop works while tenants stay alive and only the RocksDB rate-limiter budget changes.

7.2 Mechanism Checks and Alternative Controls

A fixed-budget improvement could be an artifact of a weak baseline, a stale or fragile score instance, a non-maintenance throttle, or a lower-layer control that already does the work. The checks in this subsection separate those possibilities from online cross-tenant placement.

Could equal static allocation simply be too weak a baseline? Table 3 compares against a frozen skewed allocation that helps the initial phase. Once the high-pressure set drifts, however, its overlap

falls to 1.14/4; SAKI recovers high-pressure throughput (+17.7% versus frozen skew), reduces compact bytes (-12.3%), and restores overlap to 3.14/4.

RocksDB’s local auto-tuned rate limiter addresses a different problem: each instance adapts within its configured ceiling, but unused budget is not moved across tenants. Under the same per-tenant ceilings, `static_autotuned` worsens high-pressure P99 by 28.8%, lowers high-pressure throughput by 19.7%, and has 0.00/4 high-set overlap. The same conclusion holds on the promoted higher-pressure sensitivity point: `static_autotuned` again worsens high-pressure P99 by 29.5% and lowers high-pressure throughput by 16.2% relative to static, while SAKI improves over it by 32.7% on P99 and 59.9% on throughput.

A score instance that merely chases stale debt would lag the moving high-pressure set. Pressure mode is a backlog-only online instance: it improves high-pressure P99 (-14.1%) but lags drift (overlap 1.86/4, max lag 5 windows). Demand mode adds the previous-window offered write signal, raises overlap to 3.00/4, eliminates the 5-window lag, and raises high-pressure throughput from +9.8% to +26.1%. Hybrid mode gives larger tail reductions but still lags (overlap 2.29/4, max lag 2). The ablation matches the design prediction in Section 4.5: backlog symptoms help, but offered demand is what makes the rank-and-assign loop track drift rather than stale debt.

Is the runtime actuator really a maintenance I/O knob? The four-tenant coverage microstudy in Section 5 shows measured limiter traffic rising monotonically with the configured budget and compact-plus-flush output tracking it (coverage ratio 0.99–1.02) until the workload’s maintenance demand saturates near 7.3 MB/s. This confirms the continuous actuation path used by SAKI.

Is the result tied to one fragile score instance? Section 4 treats the score $s_i = 6.00 D/1000 + 0.60 G/1000 + 0.05 T + 0.02 B + 0.02 P + 0.30 L$ as an auditable reference instance, not an optimum. First, we replay the 35 post-warmup control decisions from the five continuous main online traces using the previous-window observations that the controller saw. The replay exactly reproduces the recorded online high-budget set in all 35 decisions. The demand term alone also reproduces the reference top-four set in all 35 decisions; anchor-dominant perturbations keep exact or near-exact top-four agreement, while a pressure-only negative control matches only 10/35 decisions (Table 4). The reference score’s top-four boundary margin is positive in every replayed decision (mean 5.30, min 0.39), so the main assignment is a property of a demand-aware score family rather than an artifact of a single coefficient vector.

Second, we rerun nearby score instances on RUNTIME-DRIFT16 by perturbing anchor weight ($\times 0.5$, $\times 2$) and residual scale ($\times 0.5$, $\times 2$, $\times 0$, the last being anchor-only), with three seeds each paired with the published paper-static trials a/b/c. Table 5 shows that all five demand-aware perturbations keep the headline direction, maintain overlap at 2.86–3.00/4, and keep max lag to 0–1 window. The ANCHOR-ONLY row, where the residual scale is zero, is the runtime demand-only case: it confirms the replay result that offered demand alone fixes the high-budget set, with overlap 3.00/4 and zero lag. The residuals matter in the lower bound: anchor-only high-throughput improvement drops as low as +12.6%, whereas restoring half-strength residuals raises the minimum to +21.3%. This is consistent with the division of labor in Section 4: the demand anchor tracks the active high-pressure set, while bounded

Table 3: Mechanism checks against stronger same-budget baselines. Rows use the comparison named at left, not all rows relative to static.

Comparison	High P99	High tput	Total tput	Compact bytes	Mid tput	Overlap
SAKI vs static	-11.7%	+38.1%	-1.7%	-21.0%	-9.1%	3.14/4
static_biased vs static	-6.1%	+17.4%	-4.2%	-9.9%	-10.0%	1.14/4
static_autotuned vs static	+28.8%	-19.7%	-1.8%	-0.2%	-17.3%	0.00/4
SAKI vs static_biased	-6.0%	+17.7%	+2.8%	-12.3%	+1.1%	3.14 vs 1.14
SAKI vs static_autotuned	-31.5%	+71.9%	+0.3%	-20.8%	+10.0%	3.14 vs 0.00

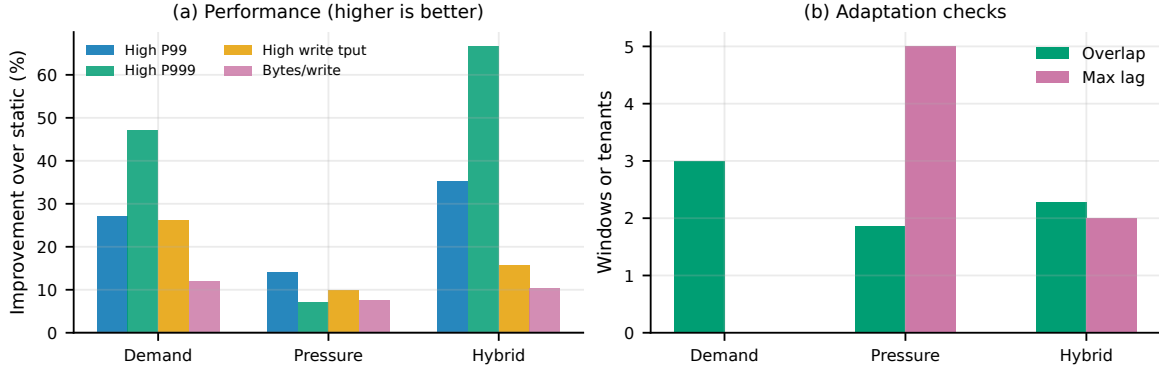


Figure 3: Score-instance ablation. Demand mode gives strong high-pressure benefit while satisfying adaptation checks; pressure and hybrid instances improve some metrics but lag the moving high-pressure set.

Table 4: Ranking-margin replay on the continuous main on-line traces (35 post-warmup decisions). Each score instance is recomputed from the previous-window observations; Jaccard compares its top-four set with the reference demand-mode top-four set.

Score instance	Exact	Jaccard	True overlap
demand only	35/35	1.00 [1.00,1.00]	3.00/4
anchor $\times 0.5$	31/35	0.95 [0.33,1.00]	2.86/4
anchor $\times 2$	35/35	1.00 [1.00,1.00]	3.00/4
residual $\times 0.5$	35/35	1.00 [1.00,1.00]	3.00/4
residual $\times 2$	31/35	0.95 [0.33,1.00]	2.86/4
pressure-only	10/35	0.61 [0.14,1.00]	2.03/4

pressure residuals appear to stabilize near-boundary tenants whose maintenance state has not fully drained. This is the empirical counterpart of Proposition 1: nearby demand-aware instances keep the same placement behavior, whereas removing demand collapses into the pressure-only case (Figure 3). Because the ablation rows pair with only three static trials, row-to-row differences should be read as consistency checks rather than tuning claims.

Finally, would a lower-layer control point be equivalent? On the aligned epoch workload, online cgroup-v2 control improves high-pressure throughput by 12.5% relative to equal cgroup throttling, but the P99 CI crosses zero (-4.4%) and compaction bytes increase by 3.1%. An oracle cgroup assignment can express priority when the high-pressure set is known (P99 -9.1%, throughput +15.4%), so

the layer is useful as an enforcement envelope. It does not provide the LSM-aware budget-placement signal that SAKI uses.

7.3 Controlled Coverage Around the Anchor Workloads

The next experiments test local coverage around the controlled anchor workloads while preserving the same-budget contract. The epoch sensitivity sweep changes drift speed and high-write pressure around EPOCH-DRIFT16. These rows are local coverage checks: the promotion rule keeps capacity-boundary and failed-tenant cases from obscuring the comparison, while the headline claim rests on the repeated anchors above. All promoted points move in the headline direction: high-pressure P99 improves by 6.9–12.9%, high-pressure throughput improves by at least 33.0%, and compaction bytes fall by 17.6–21.3%. The faster-drift point remains CI-strict on high-pressure benefit (P99 -6.9%, throughput +33.9%) with an aggregate-throughput cost (-5.0%), matching the expected responsiveness limit of a previous-window controller.

The continuous workload matrix changes one axis at a time around RUNTIME-DRIFT16: 4 KB values, tripled point-read QPS, faster drift at 1.5 tenants/window, and an 8-tenant cluster with two high-pressure tenants. All four variants were promoted to five trials and retain the headline direction: high-pressure P99 improves in every row and high-pressure throughput improves by 20.1–30.4% (Table 6). Three rows are CI-strict wins on both primary high-pressure metrics; READ-3x remains CI-strict on high throughput, but its P99 confidence interval crosses zero. The matrix also names the trade-off rather than hiding it: VALUE-4K’s P999 mean is negative

Table 5: Score-family robustness around the demand-mode SAKI instance on RUNTIME-DRIFT16 ($n=3$ per row, paired with paper-static a/b/c). Each row perturbs only the anchor or residual coefficients; all other settings are fixed. Cells show mean [min, max] percent of paired static; Top-4 Jaccard averages the post-warmup window-by-window high-set overlap against published SAKI decisions. The ^{*} row reproduces the published $n=5$ main continuous result; its static pairing covers all five trials, not the three used by the ablation rows.

Perturbation	High P99	High tput	Overlap	Max lag	Top-4 Jaccard
anchor $\times 0.5$ (3.00)	-30.3% [-38.1,-23.6]	+22.4% [+18.9,+27.1]	2.90/4	1	0.94 [0.88,1.00]
anchor $\times 2$ (12.00)	-27.5% [-42.4,-18.2]	+23.7% [+22.2,+26.2]	3.00/4	0	1.00 [1.00,1.00]
residual $\times 0.5$	-30.0% [-39.8,-16.7]	+24.3% [+21.3,+28.2]	3.00/4	0	1.00 [1.00,1.00]
residual $\times 2$	-35.8% [-49.6,-25.7]	+25.0% [+20.9,+27.1]	2.86/4	1	0.94 [0.89,1.00]
anchor only (residual= 0)	-31.9% [-40.2,-26.2]	+21.5% [+12.6,+26.0]	3.00/4	0	1.00 [1.00,1.00]
fixed SAKI ($n=5$, paper main) [*]	-27.0%	+26.1%	3.00/4	0	1.00 (self)

but not statistically separated from static, reflecting sparse heavy-compaction bursts under a conserved rate-limit ceiling rather than a P999 regression. READ-3X is a point-read-QPS perturbation, not a scan-heavy or read-dominant SLO result.

The window length is also a mechanism setting, not a tuned constant. Table 7 repeats the continuous anchor at 10, 20, and 40 seconds with three seed pairs each, holding duration, offered load, drift distance, and the 96 MB/s aggregate budget fixed. The 20-second anchor is the only point with a CI-strict high-P99 win (-26.9%); 10 seconds keeps high throughput positive but makes both P99 and throughput noisy and raises LOW P99, while 40 seconds lowers overlap to 2.00/4 and leaves adaptation lag unbounded in all three seeds. This supports using 20 seconds as a mechanism-isolation anchor and treats the other two points as responsiveness boundaries.

7.4 Collateral Cost and Liveness

SAKI is a targeted reallocation policy, not a Pareto-improvement claim. Tier-level accounting makes the redistribution explicit. In the continuous aggregate, HIGH tenants receive P99 -27.0%, P999 -47.1%, and write throughput +26.1%; LOW tenants absorb P99 +1.4%, P999 +2.6%, and write throughput -3.0%. The epoch-level pattern is similar: HIGH throughput rises by 38.1%, while MID and LOW throughput change by -9.1% and -1.2%. Total throughput remains close to static in both harnesses, so the benefit is a budget-placement trade-off rather than added capacity.

Read-side and CPU measurements show that the bytes/write improvement does not hide higher compaction CPU per completed write: CPU/write falls by 12.8% overall. The tier values are -1.6% for HIGH, -8.0% for MID, and -33.5% for LOW. Read-side effects are bounded in this read-light workload (overall read P99 +7.0%, read P999 -10.6%, read throughput +0.5%), but they are collateral accounting rather than evidence for read-dominant SLOs. A separate 360-second continuous run with 16 drift events reports high P99 -13.9%, high P999 -19.6%, high write throughput +11.6%, bytes/write -3.4%, overlap 3.00/4, max lag 1 window, and zero tenant failures—a liveness and tracking sanity check at the longer horizon.

7.5 Trace Qualification Boundary

The controlled experiments above establish the fixed-budget placement mechanism. Public traces become meaningful controller-validation workloads when they preserve tenant identity and drift, expose actuatable supply under the fixed budget, and create enough RocksDB maintenance stress to exercise compaction-budget placement. We audited two public trace families under the same 11/7/3 MB/s, 112 MB/s budget contract and use them to identify which of those conditions are missing.

Meta CacheLib kvcache/202401 [17] is useful as a trace-shaped replay and tracking smoke, but it records cache-front-end requests rather than RocksDB compaction events. Five selected drifting 16-tenant segments completed with zero failed tenants and preserved drift, yet the replay failed the engine-stress gate: offered writes were 110–143 MB/s, completed logical writes were only 6.7–7.3 MB/s, compaction output plateaued near 29–31 MB/s, and no segment entered the stress-pass subset. Meta Tectonic/Baleen storage 0.1% [32] exposes the complementary condition. Aggregate scaled write supply initially looked sufficient, but most windows have only one to three active write tenants. Under the static all-mid policy, the actuatable supply $\sum_i \min(7, \text{traceWrite}_i)$ is far below the aggregate trace rate; across 146 independent candidates, zero have six or more windows above 70 MB/s or mean static-actuatable supply above 70 MB/s. These audits are therefore trace-qualification results rather than controller comparisons.

8 DISCUSSION AND LIMITATIONS

8.1 Applicability Conditions

SAKI applies when co-located RocksDB instances share a fixed background budget, write pressure drifts across tenants at window time scales, and the host can observe tenant-local demand and maintenance symptoms. The repeated epoch runs establish the main placement effect, the continuous runs show that it survives runtime rate-limiter actuation, and the mechanism checks rule out stale skew, tenant-local autotuning, pressure-only tracking, lower-layer throttling, and a non-maintenance actuation path. Ranking-margin replay and perturbation runs further show that the main continuous high-budget assignment is fixed by a demand-aware score family rather than by one isolated coefficient vector. Together,

Table 6: Local workload matrix, five trials per variant, online vs. static (percent changes, 95% CI half-widths in pp). Zero failed tenants in all runs.

Variant	High P99	High P999	High tput	Total tput	Bytes/write	Overlap
VALUE-4K	-28.6%±16.2	-7.1%±20.6	+20.1%±4.7	+0.6%±1.5	-9.0%±3.0	3.00/4
READ-3X	-15.4%±17.5	-38.4%±27.6	+30.4%±7.6	-0.2%±4.0	-10.2%±5.8	3.00/4
FAST-DRIFT	-17.6%±10.1	-1.1%±46.6	+20.5%±10.3	+4.4%±3.4	-3.3%±4.4	2.57/4
TENANT-8	-42.2%±12.7	-52.6%±16.9	+28.7%±5.8	+7.0%±1.9	-11.3%±3.4	1.57/2

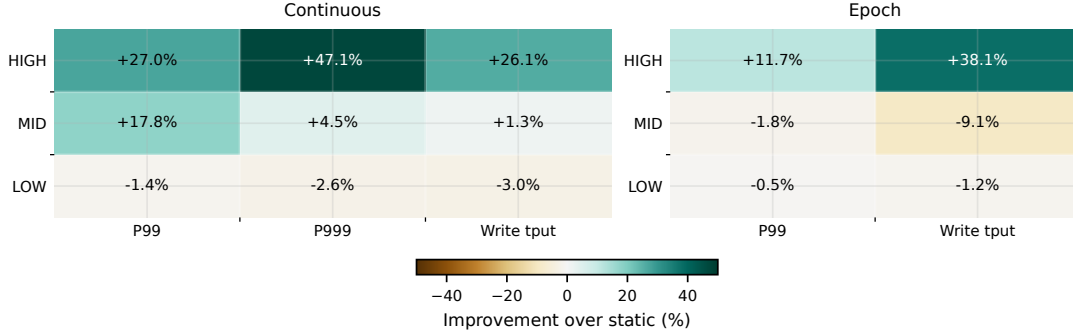


Figure 4: Tier-wise benefit and collateral cost. Cells report SAKI's improvement over static (positive is better), showing the intended high-pressure benefit and the measured redistribution cost to lower tiers.

Table 7: Window-length sensitivity around the continuous anchor (three seed pairs per row, online vs. static; CI half-widths in pp).

Window	High P99	High tput	Overlap	LOW P99
10 s	+6.8%±39.2	+23.4%±58.1	3.47/4	+110.5%±367.8
20 s	-26.9%±14.0	+16.1%±8.7	3.00/4	+24.6%±63.6
40 s	-14.4%±44.8	+14.5%±19.8	2.00/4	-36.3%±84.2

these results support the systems claim that a host-level rank-and-assign controller can place an existing compaction/I/O budget better than equal sharing.

The bounded-drift condition matters. SAKI relies on previous-window observations remaining relevant to the next window. The main continuous workload moves the high-pressure set by one tenant per 20-second window, and FAST-DRIFT pushes this to 1.5 tenants/window while retaining CI-strict high-pressure benefit with overlap 2.57/4. Abrupt reversals, sub-window bursts, and within-window phase changes may outrun the controller. The 20-second window, 4/8/4 tier split, and concrete budget levels are mechanism-isolation settings. In the continuous anchor, a 10/20/40-second window sweep keeps the 96 MB/s budget fixed and brackets the trade-off: 10 seconds improves assignment frequency but makes high-P99 and LOW collateral noisier, whereas 40 seconds reduces overlap and leaves adaptation unbounded in all three seeds. The conservation rule in Section 4 supports other tier counts, budget spreads, or a continuous allocator.

8.2 Deployment Assumptions

The continuous harness changes background rate limits through `RateLimiter::SetBytesPerSecond()`. The measured rate-limiter coverage microstudy shows this knob shapes flush/compaction I/O in our configuration, so foreground latency and throughput changes arise indirectly from maintenance service rather than direct foreground throttling. RocksDB's built-in auto-tuned rate limiter remains useful local machinery, but it adapts within per-tenant ceilings; SAKI supplies the host-level loop that moves unused budget across tenants. A natural deployment variant is to let SAKI choose tenant ceilings while RocksDB autotuning adapts inside each ceiling; we did not evaluate that combined policy.

A production deployment would need tenant inventory, periodic RocksDB statistics, a conserved host budget, and per-tenant runtime actuation. The offered-demand signal should be measured near admission, such as at a client proxy, RPC ingress path, tenant queue, or write-admission counter, before upstream backpressure, drops, or retries hide true pressure. Operators would likely add smoothing, hysteresis, cooldowns, minimum dwell times, and SLO-aware floors around the simple rank-and-assign rule. If metrics or tenant inventory are missing, the safe operational choice is to retain the last valid assignment or fall back to all-mid sharing, preserving the aggregate budget before re-enabling reallocation. The reference score should be treated as a demand-aware instance of the framework, not as a portable constant vector: deployments with different admission signals or maintenance dynamics should renormalize terms while preserving budget conservation and demand observability. Other LSM engines need analogous tenant-scoped maintenance signals and a runtime maintenance-budget knob.

8.3 Resource and Fairness Boundaries

SAKI controls one resource dimension: background I/O budget placement. CPU, cache, memory, device queueing, and per-level write amplification may become the limiting resources in other deployments. A deployment can detect this regime when rate-limiter traffic stops tracking flush/compaction output or when CPU, cache, or device-queue metrics saturate while compaction debt remains high. The continuous efficiency metric is therefore normalized bytes/write because SAKI completes more foreground writes; a full maintenance-cost decomposition is a separate multi-resource control problem.

The policy is also a targeted reallocation. Tier-wise reporting makes the collateral cost explicit because shifting budget to high-pressure tenants can cost lower-pressure tenants. In the main continuous aggregate, LOW tenants see small write-tail and throughput costs while HIGH tenants improve substantially; the 4 KB-value workload leaves P999 not CI-separated from zero despite a CI-strict P99 win. These results motivate SLO-aware floors and fairness policy as deployment layers around the same fixed-budget loop. The read-heavy variant raises point-read QPS around the main harness; scan-heavy workloads, range-scan SLOs, and read-dominant deployments require their own scoring and validation layer.

8.4 Trace Validation Boundary

The public-trace audits define the conditions required for meaningful controller validation. CacheLib provides real cache-front-end request traces and validates trace-shaped replay plumbing, but the selected segments create too little RocksDB maintenance stress: completed logical writes were only about 5–6% of offered demand and compaction output plateaued near 30 MB/s. Baleen exposes a different issue: aggregate scaled write supply can look adequate while too few tenants are active in each window to actuate the fixed all-mid budget. Across 146 independent candidates, no segment had enough static-actuatable supply to justify a second static smoke.

The next production-trace validation layer needs storage-engine traces with tenant identity, drift, sustained write pressure, and actuatable supply under the same fixed-budget contract. Non-leveled compaction styles, longer deployments, and joint multi-resource control are natural extensions after that trace condition is met.

9 RELATED WORK

LSM compaction and tuning. The original LSM-tree design buffers updates and later merges them into disk-resident sorted components [22]. The survey by Luo and Carey describes the resulting design space [16], and Sarkar et al. formalize compaction triggers, layout, granularity, and data movement as design primitives [24]. Monkey and Dostoevsky optimize Bloom-filter memory, merge policy, and space-time tradeoffs [4, 5]; WiscKey and PebblesDB change value placement or merge structure [15, 23]. bLSM, TRIAD, and Accordion improve write smoothing, logging, and memory organization [1, 3, 25]. SILK schedules internal LSM I/O to reduce latency spikes, while ADOC, vLSM, and ArceKV tune dataflow, LSM shape, or compaction thresholds for a single engine or workload [2, 14, 33, 34]. Dremel adaptively tunes RocksDB parameters with an online bandit on a single instance [36], and Dong et al.’s RocksDB retrospective at Meta tracks how amplification priorities

shifted with hardware and workload, still inside one engine [7]. SAKI is orthogonal: it leaves each engine’s compaction policy and parameters unchanged, treats these single-engine contributions as the inner control loop, and asks how a fixed host budget should be divided across co-located instances as demand drifts. The two control layers can be combined.

Multi-tenant storage control. Argon, PARDA, mClock, Pisces, Cake, and IOFlow allocate or route storage service across shared servers, hypervisors, and software-defined storage planes [9, 10, 26, 28–30]. Linux cgroups and IOCost provide lower-layer block-I/O weighting [11, 27], and Gimbal extends the line to SmartNIC JBOFs that schedule co-tenant NVMe traffic on the storage data path [20]. These systems enforce fairness or SLOs over I/O streams but do not observe LSM debt signals or choose which RocksDB instance receives background compaction service. SAKI uses those engine-level signals and treats kernel or network-side I/O control as a companion enforcement layer rather than the placement mechanism itself.

Multi-tenant LSM deployments. Recent KV-store work moves isolation closer to the storage engine. FlexEngine uses admission control for multi-tenant serverless LSM deployments, Iso-KVSSD and ZenFS+ use device namespace or zoned-placement mechanisms for isolation, and Delta Fair Sharing proposes in-engine multi-tenant fairness for RocksDB [8, 19, 21, 31]. CaaS-LSM and DFlush move compaction or flush work into disaggregated or DPU-assisted settings [6, 35]. SAKI instead targets the common deployment where tenants are separate, unmodified RocksDB instances on one host and the available background budget is fixed.

RocksDB controls. RocksDB exposes per-instance options, statistics, event listeners, and rate limiting [13, 18]. The auto-tuned rate limiter solves local adaptation within a ceiling; SAKI supplies the host-level rank-and-assign loop that places budgets across instances under a conserved aggregate budget. Public CacheLib and Baleen traces inform our trace-qualification audits, but cache admission, prefetching, or request-trace replay are separate from the fixed-budget RocksDB compaction-placement problem [17, 32]. SAKI reallocates a conserved host budget across separate instances without changing the engine.

10 CONCLUSION

SAKI addresses multi-tenant RocksDB compaction-debt mismatch with an external conserved rank-and-assign controller that reallocates per-tenant background budgets from previous-window observations while preserving the aggregate host budget. A demand-observability model explains why demand-aware score instances track bounded drift and why pressure-only ranking can chase stale debt. Repeated epoch and continuous runs show high-pressure write-tail and throughput gains; stronger baselines, score ablations, ranking-margin replay, cgroup-v2, and rate-limiter coverage checks tie those gains to host-level placement of a real maintenance-I/O knob. Workload variants, collateral accounting, and trace qualification define the current evidence boundary. For bounded write-heavy drift under fixed shared maintenance budgets, SAKI shows that shared compaction budgeting is not merely a configuration detail but a host-level control point.

REFERENCES

- [1] Oana Balmau, Diego Didona, Rachid Guerraoui, Willy Zwaenepoel, Huapeng Yuan, Aashray Arora, Karan Gupta, and Pavan Konka. 2017. TRIAD: Creating Synergies Between Memory, Disk and Log in Log Structured Key-Value Stores. In *2017 USENIX Annual Technical Conference (USENIX ATC '17)*. USENIX Association, Santa Clara, CA, USA, 363–375.
- [2] Oana Balmau, Diego Didona, Rachid Guerraoui, Willy Zwaenepoel, Huapeng Yuan, Aakash Arora, Karan Gupta, and Pavan Konka. 2019. SILK: Preventing Latency Spikes in Log-Structured Merge Key-Value Stores. In *2019 USENIX Annual Technical Conference (USENIX ATC '19)*. USENIX Association, Renton, WA, USA, 753–766.
- [3] Edward Bortnikov, Anastasia Braginsky, Eshcar Hillel, Idit Keidar, and Gali Sheffi. 2018. Accordion: Better Memory Organization for LSM Key-Value Stores. *Proceedings of the VLDB Endowment* 11, 12 (2018), 1863–1875. <https://doi.org/10.14778/3229863.3229873>
- [4] Niv Dayan, Manos Athanassoulis, and Stratos Idreos. 2017. Monkey: Optimal Navigable Key-Value Store. In *Proceedings of the 2017 ACM International Conference on Management of Data (SIGMOD '17)*. Association for Computing Machinery, New York, NY, USA, 79–94. <https://doi.org/10.1145/3035918.3064054>
- [5] Niv Dayan and Stratos Idreos. 2018. Dostoevsky: Better Space-Time Trade-Offs for LSM-Tree Based Key-Value Stores via Adaptive Removal of Superfluous Merging. In *Proceedings of the 2018 International Conference on Management of Data (SIGMOD '18)*. Association for Computing Machinery, New York, NY, USA, 505–520. <https://doi.org/10.1145/3183713.3196927>
- [6] Chen Ding, Kai Lu, Quanyi Zhang, Zekun Ye, Ting Yao, Daohui Wang, Huatao Wu, and Jiguang Wan. 2025. DFlush: DPU-Offloaded Flush for Disaggregated LSM-based Key-Value Stores. *Proceedings of the ACM on Management of Data* 3, 3, Article 147 (2025), 28 pages. <https://doi.org/10.1145/3725284>
- [7] Siying Dong, Andrew Kryczka, Yanqin Jin, and Michael Stumm. 2021. RocksDB: Evolution of Development Priorities in a Key-value Store Serving Large-scale Applications. *ACM Transactions on Storage* 17, 4, Article 26 (October 2021), 32 pages. <https://doi.org/10.1145/3483840>
- [8] Tyler Griggs, Soujanya Ponnappalli, Dev Bali, Wenjie Ma, James DeLoye, Audrey Cheng, Jaewan Hong, Natacha Crooks, Scott Shenker, Ion Stoica, and Matei Zaharia. 2026. Delta Fair Sharing: Performance Isolation for Multi-Tenant Storage Systems. [arXiv:2601.20030](https://arxiv.org/abs/2601.20030). <https://doi.org/10.48550/arXiv.2601.20030>
- [9] Ajay Gulati, Irfan Ahmad, and Carl A. Waldspurger. 2009. PARDA: Proportional Allocation of Resources for Distributed Storage Access. In *7th USENIX Conference on File and Storage Technologies (FAST '09)*. USENIX Association, San Francisco, CA, USA, 85–98.
- [10] Ajay Gulati, Arif Merchant, and Peter J. Varman. 2010. mClock: Handling Throughput Variability for Hypervisor IO Scheduling. In *9th USENIX Symposium on Operating Systems Design and Implementation (OSDI '10)*. USENIX Association, Vancouver, BC, Canada, 437–450.
- [11] Tejun Heo, Dan Schatzberg, Andrew Newell, Song Liu, Saravanan Dhakshinamurthy, Iyyswarya Narayanan, Josef Bacik, Chris Mason, Chunqiang Tang, and Dimitrios Skarlatos. 2022. IOCost: Block IO Control for Containers in Datacenters. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '22)*. Association for Computing Machinery, New York, NY, USA, 595–608. <https://doi.org/10.1145/3503222.3507727>
- [12] Shubham Kaushik and Subhadeep Sarkar. 2024. Anatomy of the LSM Memory Buffer: Insights & Implications. In *Proceedings of the 10th International Workshop on Testing Database Systems (DBTest '24)*. Association for Computing Machinery, New York, NY, USA. <https://doi.org/10.1145/3662165.3662766>
- [13] Andrew Kryczka. 2017. Auto-tuned Rate Limiter. <https://rocksdb.org/blog/2017/12/18/17-auto-tuned-rate-limiter.html>. RocksDB blog, accessed 2026-05-14.
- [14] Junfeng Liu, Haoxuan Xie, and Siqiang Luo. 2026. ArceKV: Towards Workload-driven LSM-compactions for Key-Value Store Under Dynamic Workloads. *Proceedings of the VLDB Endowment* 19, 5 (2026), 958–972. <https://doi.org/10.14778/3796195.3796208>
- [15] Lanyue Lu, Thanumalayan Sankaranarayanan Pillai, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. 2016. WiscKey: Separating Keys from Values in SSD-conscious Storage. In *14th USENIX Conference on File and Storage Technologies (FAST '16)*. USENIX Association, Renton, WA, USA, 133–148.
- [16] Chen Luo and Michael J. Carey. 2020. LSM-based Storage Techniques: A Survey. *The VLDB Journal* 29, 1 (2020), 393–418.
- [17] Meta Platforms, Inc. 2024. CacheLib Public Cache Trace Dataset. https://github.com/cacheMon/cache_dataset. Meta CacheLib kvcache/202401 traces, accessed 2026-05-18.
- [18] Meta Platforms, Inc. 2026. RocksDB. <https://rocksdb.org/>. Accessed 2026-05-14.
- [19] Donghyun Min and Youngjae Kim. 2021. Isolating Namespace and Performance in Key-Value SSDs for Multi-tenant Environments. In *Proceedings of the 36th Annual ACM Symposium on Applied Computing*. Association for Computing Machinery, New York, NY, USA, 1112–1121. <https://doi.org/10.1145/3412841.3442008>
- [20] Jaehong Min, Ming Liu, Tapan Chugh, Chenxingyu Zhao, Patrick Wei, In Hwan Doh, and Arvind Krishnamurthy. 2021. Gimbal: Enabling Multi-tenant Storage Disaggregation on SmartNIC JBOfs. In *Proceedings of the 2021 ACM SIGCOMM Conference (SIGCOMM '21)*. Association for Computing Machinery, New York, NY, USA, 106–122. <https://doi.org/10.1145/3452296.3472940>
- [21] Myoungsoon Oh, Seehwan Yoo, Jongmoo Choi, J. Park, and C. E. Choi. 2023. ZenFS+: Nurturing Performance and Isolation to ZenFS. *IEEE Access* 11 (2023), 26344–26357. <https://doi.org/10.1109/ACCESS.2023.3257354>
- [22] Patrick O’Neil, Edward Cheng, Dieter Gawlick, and Elizabeth O’Neil. 1996. The Log-Structured Merge-Tree (LSM-Tree). *Acta Informatica* 33, 4 (1996), 351–385. <https://doi.org/10.1007/s002360050048>
- [23] Pandian Raju, Rohan Kadekodi, Vijay Chidambaram, and Ittai Abraham. 2017. PebblesDB: Building Key-Value Stores using Fragmented Log-Structured Merge Trees. In *Proceedings of the 26th Symposium on Operating Systems Principles (SOSP '17)*. Association for Computing Machinery, New York, NY, USA, 497–514. <https://doi.org/10.1145/3132747.3132765>
- [24] Subhadeep Sarkar, Dimitris Staratzis, Zichen Zhu, and Manos Athanassoulis. 2021. Constructing and Analyzing the LSM Compaction Design Space. *Proceedings of the VLDB Endowment* 14, 11 (2021), 2216–2229. <https://doi.org/10.14778/3476249.3476274>
- [25] Russell Sears and Raghu Ramakrishnan. 2012. bLSM: A General Purpose Log Structured Merge Tree. In *Proceedings of the 2012 ACM SIGMOD International Conference on Management of Data (SIGMOD '12)*. Association for Computing Machinery, New York, NY, USA, 217–228. <https://doi.org/10.1145/2213836.2213862>
- [26] David Shue, Michael J. Freedman, and Anees Shaikh. 2012. Performance Isolation and Fairness for Multi-Tenant Cloud Storage. In *10th USENIX Symposium on Operating Systems Design and Implementation (OSDI '12)*. USENIX Association, Hollywood, CA, USA, 349–362.
- [27] The Linux Kernel Developers. 2026. Control Group v2. <https://www.kernel.org/doc/html/latest/admin-guide/cgroup-v2.html>. Accessed 2026-05-15.
- [28] Eno Thereska, Hitesh Ballani, Greg O’Shea, Thomas Karagiannis, Ant Rowstron, Tom Talpey, Richard Black, and Timothy Zhu. 2013. IOFlow: A Software-Defined Storage Architecture. In *Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles (SOSP '13)*. Association for Computing Machinery, New York, NY, USA, 182–196. <https://doi.org/10.1145/2517349.2522723>
- [29] Matthew Wachs, Michael Abd-El-Malek, Eno Thereska, and Gregory R. Ganger. 2007. Argon: Performance Isolation for Shared Storage Servers. In *5th USENIX Conference on File and Storage Technologies (FAST '07)*. USENIX Association, San Jose, CA, USA, 61–76.
- [30] Andrew Wang, Shivaram Venkataraman, Sara Alspaugh, Randy H. Katz, and Ion Stoica. 2012. Cake: Enabling High-Level SLOs on Shared Storage Systems. In *Proceedings of the Third ACM Symposium on Cloud Computing (SoCC '12)*. Association for Computing Machinery, New York, NY, USA, Article 14, 14 pages. <https://doi.org/10.1145/2391229.2391243>
- [31] Yingjia Wang, Caixin Gong, Guoyun Zhu, Sheng Wang, Zhengheng Wang, Huan Liu, Junzhi Shi, Wu Qin, Wei Zhang, Feifei Li, and Ming-Chang Yang. 2026. Making LSM-Tree-based Key-Value Store Practical and Efficient for Multi-Tenant Serverless Cloud Databases. *Proceedings of the ACM on Management of Data* 4, 1, Article 53 (February 2026), 25 pages. <https://doi.org/10.1145/3786667>
- [32] Daniel Lin-Kit Wong, Hao Wu, Carson Molder, Sathya Gunasekar, Jimmy Lu, Snehal Khandkar, Abhinav Sharma, Daniel S. Berger, Nathan Beckmann, and Gregory R. Ganger. 2024. Baleen: ML Admission & Prefetching for Flash Caches. In *22nd USENIX Conference on File and Storage Technologies (FAST '24)*. USENIX Association, Santa Clara, CA, 347–371. <https://www.usenix.org/conference/fast24/presentation/wong>
- [33] Giorgos Xanthakis, Antonios Katsarakis, Giorgos Saloustros, and Angelos Bilas. 2024. vLSM: Low Tail Latency and I/O Amplification in LSM-based KV Stores. [arXiv:2407.15581](https://arxiv.org/abs/2407.15581). <https://doi.org/10.48550/arXiv.2407.15581>
- [34] Jinghuan Yu, Sam H. Noh, Young-ri Choi, and Chun Jason Xue. 2023. ADOC: Automatically Harmonizing Dataflow Between Components in Log-Structured Key-Value Stores for Improved Performance. In *21st USENIX Conference on File and Storage Technologies (FAST '23)*. USENIX Association, Santa Clara, CA, USA, 65–80.
- [35] Qiaolin Yu, Chang Guo, Jay Zhuang, Viraj Thakkar, Jianguo Wang, and Zhichao Cao. 2024. CaaS-LSM: Compaction-as-a-Service for LSM-based Key-Value Stores in Storage Disaggregated Infrastructure. *Proceedings of the ACM on Management of Data* 2, 3, Article 124 (2024), 28 pages. <https://doi.org/10.1145/3654927>
- [36] Chenxingyu Zhao, Tapan Chugh, Jaehong Min, Ming Liu, and Arvind Krishnamurthy. 2022. Dremel: Adaptive Configuration Tuning of RocksDB KV-Store. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* 6, 2, Article 33 (2022), 30 pages. <https://doi.org/10.1145/3530903>